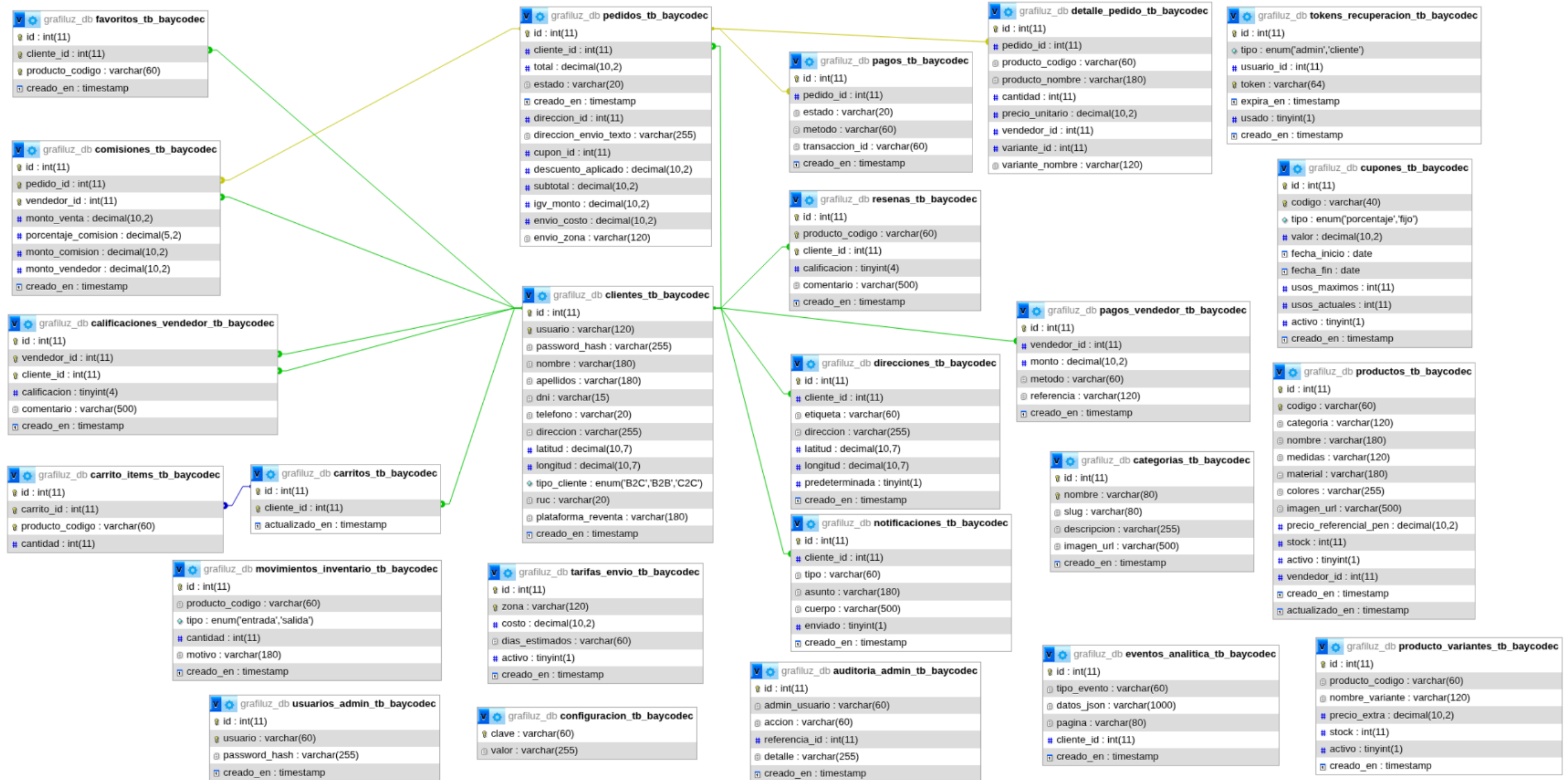


## Es un diagrama de entidad-relación (ER)



**Gestión de Usuarios y Clientes:** La tabla central es `clientes_tb_baycodec`, que se relaciona con direcciones (`direcciones_tb_baycodec`), notificaciones, carritos de compras, reseñas y pedidos. También cuenta con tablas para administración (`usuarios_admin_tb_baycodec`, `auditoria_admin_tb_baycodec`) y recuperación de tokens.

**Catálogo de Productos:** Gestionado mediante `productos_tb_baycodec` (con información de precios, stock, categorías) y complementado por `producto_variantes_tb_baycodec` para variantes específicas de productos, además de la tabla de `categorias_tb_baycodec`.

**Proceso de Venta y Carrito:** Las tablas `carritos_tb_baycodec` y `carrito_items_tb_baycodec` manejan la selección previa a la compra. Los pedidos se consolidan en `pedidos_tb_baycodec` y sus respectivos ítems en `detalle_pedido_tb_baycodec`.

**Logística y Envíos:** Incluye tablas para tarifas de envío (`tarifas_envio_tb_baycodec`) y movimientos de inventario (`movimientos_inventario_tb_baycodec`).

**Pagos y Finanzas:** Registro de transacciones mediante `pagos_tb_baycodec`, pagos a vendedores (`pagos_vendedor_tb_baycodec`), comisiones y cupones de descuento (`cupones_tb_baycodec`).

**Reputación y Analítica:** Tablas de calificaciones y reseñas tanto para productos como para vendedores (`calificaciones_vendor_tb_baycodec`, `resenas_tb_baycodec`), además de un registro de eventos de analítica (`eventos_analitica_tb_baycodec`).

<https://www.planttext.com/>

```
@startuml
skinparam classAttributeIconSize 0
skinparam backgroundColor #FEFEFE
```

```
class clientes_tb_baycodec {
+ id : int(11)
+ usuario : varchar(120)
+ password_hash : varchar(255)
+ nombre : varchar(180)
+ apellidos : varchar(180)
+ dni : varchar(15)
+ telefono : varchar(20)
+ direccion : varchar(255)
+ latitud : decimal(10,7)
+ longitud : decimal(10,7)
+ tipo_cliente : enum('B2C','B2B','C2C')
+ ruc : varchar(20)
+ plataforma_reventa : varchar(180)
+ creado_en : timestamp
}
```

```
class direcciones_tb_baycodec {
+ id : int(11)
+ cliente_id : int(11)
+ etiqueta : varchar(60)
+ direccion : varchar(255)
+ latitud : decimal(10,7)
+ longitud : decimal(10,7)
+ predeterminada : tinyint(1)
+ creado_en : timestamp
}
```

```
class carritos_tb_baycodec {
+ id : int(11)
+ cliente_id : int(11)
+ actualizado_en : timestamp
}
```

```
class carrito_items_tb_baycodec {
+ id : int(11)
+ carrito_id : int(11)
+ producto_codigo : varchar(60)
+ cantidad : int(11)
}
```

```
class pedidos_tb_baycodec {
+ id : int(11)
+ cliente_id : int(11)
+ total : decimal(10,2)
+ estado : varchar(20)
+ creado_en : timestamp
+ direccion_id : int(11)
+ direccion_envio_texto : varchar(255)
+ cupon_id : int(11)
+ descuento_aplicado : decimal(10,2)
+ subtotal : decimal(10,2)
+ igv_monto : decimal(10,2)
+ envio_costo : decimal(10,2)
+ envio_zona : varchar(120)
}
```

```
class detalle_pedido_tb_baycodec {
+ id : int(11)
+ pedido_id : int(11)
+ producto_codigo : varchar(60)
+ producto_nombre : varchar(180)
+ cantidad : int(11)
+ precio_unitario : decimal(10,2)
+ vendedor_id : int(11)
+ variante_id : int(11)
+ variante_nombre : varchar(120)
}
```

```
class pagos_tb_baycodec {
+ id : int(11)
+ pedido_id : int(11)
+ estado : varchar(20)
+ metodo : varchar(60)
+ transaccion_id : varchar(60)
+ creado_en : timestamp
}
```

```
class productos_tb_baycodec {
+ id : int(11)
+ codigo : varchar(60)
+ categoria : varchar(120)
+ nombre : varchar(180)
+ medidas : varchar(120)
+ material : varchar(180)
+ colores : varchar(255)
```

```
+ imagen_url : varchar(500)
+ precio_referencial_pen : decimal(10,2)
+ stock : int(11)
+ activo : tinyint(1)
+ vendedor_id : int(11)
+ creado_en : timestamp
+ actualizado_en : timestamp
}
```

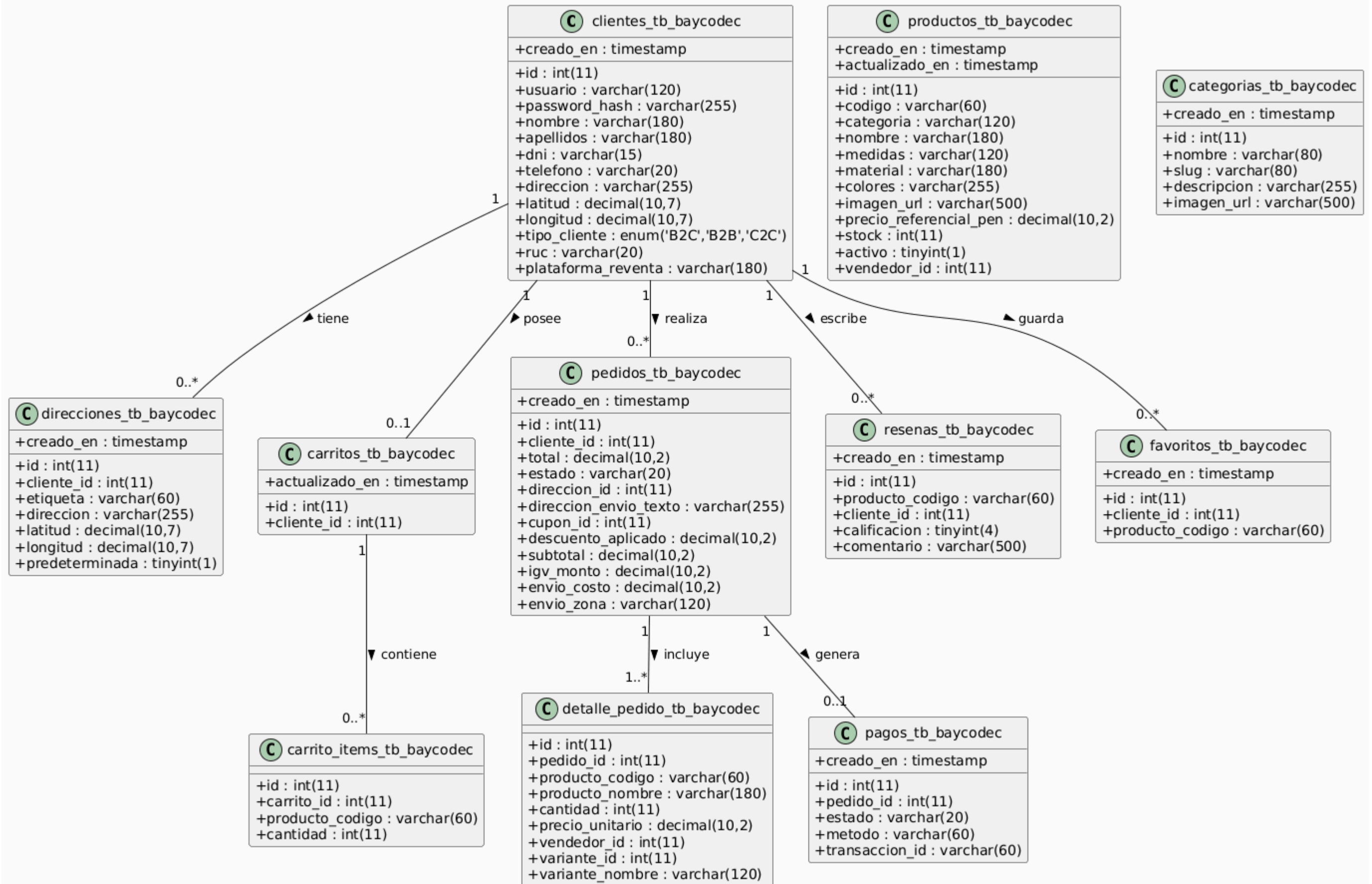
```
class categorias_tb_baycodec {
+ id : int(11)
+ nombre : varchar(80)
+ slug : varchar(80)
+ descripcion : varchar(255)
+ imagen_url : varchar(500)
+ creado_en : timestamp
}
```

```
class resenas_tb_baycodec {
+ id : int(11)
+ producto_codigo : varchar(60)
+ cliente_id : int(11)
+ calificacion : tinyint(4)
+ comentario : varchar(500)
+ creado_en : timestamp
}
```

```
class favoritos_tb_baycodec {
+ id : int(11)
+ cliente_id : int(11)
+ producto_codigo : varchar(60)
+ creado_en : timestamp
}
```

```
' Relaciones principales
clientes_tb_baycodec "1" -- "0..*" direcciones_tb_baycodec : tiene >
clientes_tb_baycodec "1" -- "0..1" carritos_tb_baycodec : posee >
carritos_tb_baycodec "1" -- "0..*" carrito_items_tb_baycodec : contiene >
clientes_tb_baycodec "1" -- "0..*" pedidos_tb_baycodec : realiza >
pedidos_tb_baycodec "1" -- "1..*" detalle_pedido_tb_baycodec : incluye >
pedidos_tb_baycodec "1" -- "0..1" pagos_tb_baycodec : genera >
clientes_tb_baycodec "1" -- "0..*" resenas_tb_baycodec : escribe >
clientes_tb_baycodec "1" -- "0..*" favoritos_tb_baycodec : guarda >
@enduml
```

## Diagrama de Clases (UML)



## Diagrama de Componentes (Component Diagram)

```
@startuml
skinparam backgroundColor #FEFEFE
skinparam componentStyle rectangle

package "Sistema E-Commerce (grafiluz_db)" {

    [Módulo de Autenticación y Seguridad] as Auth
    [Módulo de Clientes y Perfiles] as ClientesMod
    [Módulo de Catálogo y Productos] as Catalogo
    [Módulo de Carrito y Checkout] as Checkout
    [Módulo de Logística e Inventario] as Logistica
    [Módulo de Pagos y Finanzas] as Finanzas
    [Módulo de Administración y Auditoría] as AdminMod

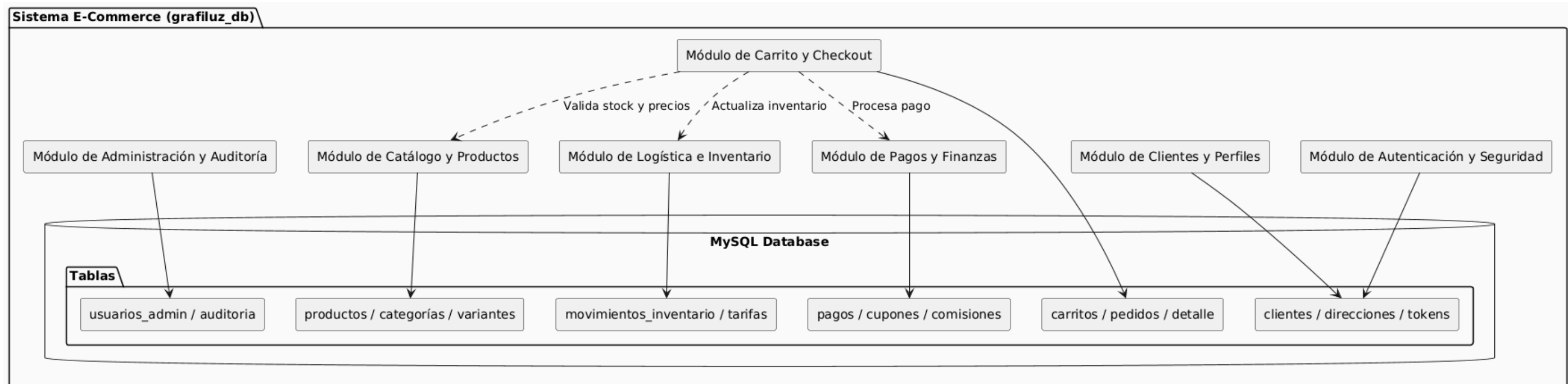
}
```

```
database "MySQL Database" {
    folder "Tablas" {
        [clientes / direcciones / tokens]
        [productos / categorías / variantes]
        [carritos / pedidos / detalle]
        [movimientos_inventario / tarifas]
        [pagos / cupones / comisiones]
        [usuarios_admin / auditoria]
    }
}

' Relaciones de los componentes con la base de datos
Auth --> [clientes / direcciones / tokens]
```

```
ClientesMod --> [clientes / direcciones / tokens]
Catalogo --> [productos / categorías / variantes]
Checkout --> [carritos / pedidos / detalle]
Logistica --> [movimientos_inventario / tarifas]
Finanzas --> [pagos / cupones / comisiones]
AdminMod --> [usuarios_admin / auditoria]

' Interacciones entre módulos
Checkout ..> Catalogo : Valida stock y precios
Checkout ..> Finanzas : Procesa pago
Checkout ..> Logistica : Actualiza inventario
@enduml
```



## Diagrama de Despliegue (Deployment Diagram)

@startuml

skinparam backgroundColor #FEFEFE

left to right direction

```
node "Dispositivo del Cliente" {  
    [Navegador Web / App Móvil] as ClientApp  
}
```

```
node "Servidor Web / Aplicación (Backend)" {  
    [API REST / Backend Node.js/PHP/Python] as BackendApp  
}
```

```
node "Servidor de Base de Datos" {  
    database "MySQL Server" {  
        [grafiluz_db]  
    }  
}
```

ClientApp --> BackendApp : HTTP / HTTPS (JSON)

BackendApp --> [grafiluz\_db] : Conexión JDBC / PDO / MySQL (TCP/IP)

@enduml



## Diagrama de Casos de Uso (Use Case Diagram)

@startuml

skinparam backgroundColor #FEFEFE

left to right direction

actor "Cliente" as Cliente

actor "Vendedor" as Vendedor

actor "Administrador" as Admin

```
rectangle "Sistema E-Commerce (grafiluz_db)" {
    usecase "Registrarse / Iniciar Sesión" as UC1
    usecase "Gestionar Catálogo de Productos" as UC2
    usecase "Gestionar Carrito de Compras" as UC3
    usecase "Realizar Pedido y Checkout" as UC4
    usecase "Procesar Pagos" as UC5
    usecase "Gestionar Inventario y Stock" as UC6
    usecase "Dejar Reseñas y Calificaciones" as UC7
    usecase "Gestionar Cupones y Descuentos" as UC8
    usecase "Revisar Auditoría y Reportes" as UC9
}
```

' Relaciones del Cliente

Cliente --> UC1

Cliente --> UC3

Cliente --> UC4

Cliente --> UC7

' Relaciones del Vendedor

Vendedor --> UC1

Vendedor --> UC2

Vendedor --> UC6

' Relaciones del Administrador

Admin --> UC1

Admin --> UC2

Admin --> UC5

Admin --> UC8

Admin --> UC9

@enduml

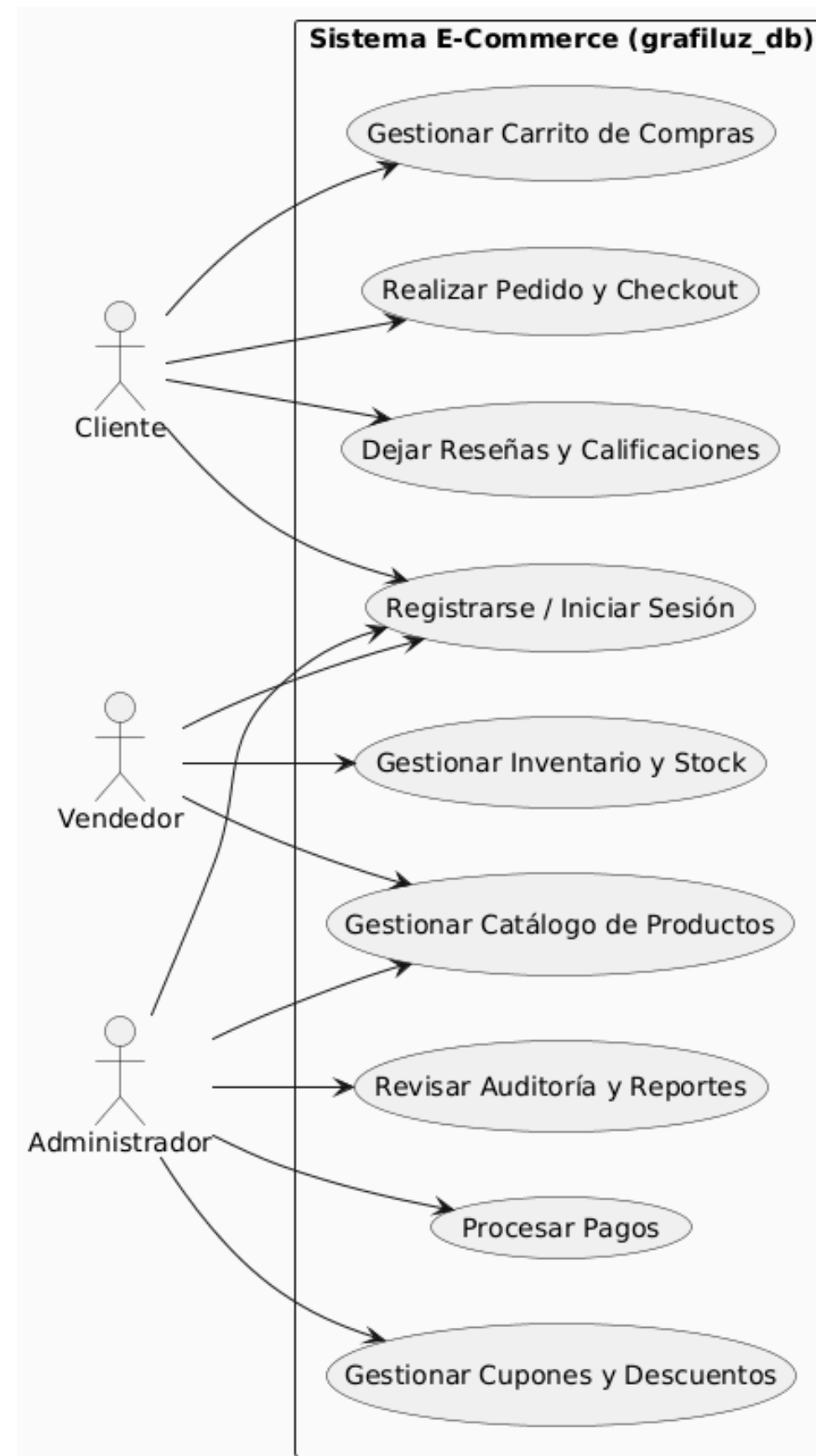




Diagrama de Secuencia (Sequence Diagram)

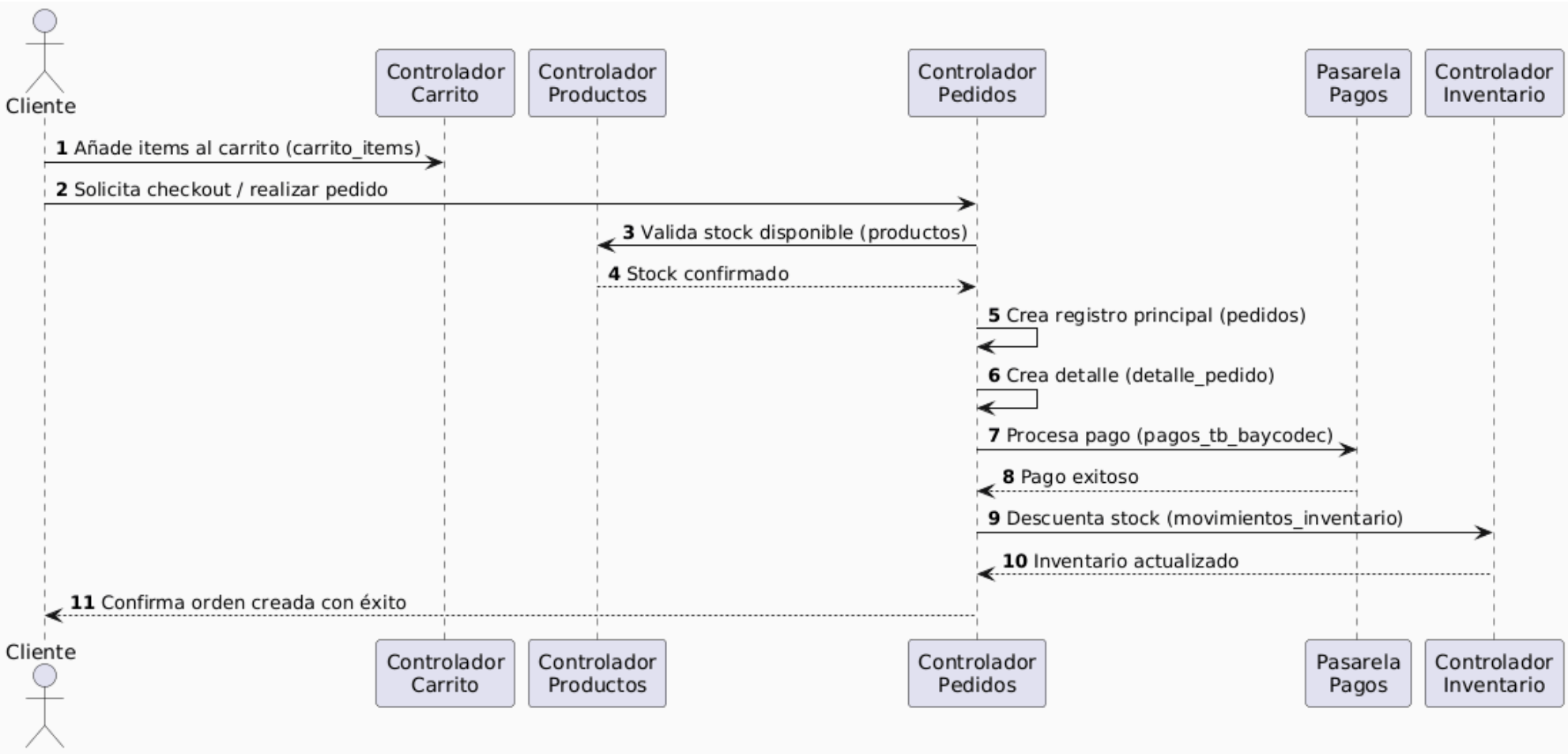
```
@startuml
skinparam backgroundColor #FEFEFE
autonumber

actor Cliente
participant "Controlador\nCarrito" as Carrito
participant "Controlador\nProductos" as Productos
participant "Controlador\nPedidos" as Pedidos
participant "Pasarela\nPagos" as Pagos
participant "Controlador\nInventario" as Inventario

Cliente->>Carrito: 1 Añade items al carrito (carrito_items)
Cliente->>Pedidos: 2 Solicita checkout / realizar pedido
Pedidos->>Productos: 3 Valida stock disponible (productos)
Productos-->>Pedidos: 4 Stock confirmado
Pedidos->>Pedidos: 5 Crea registro principal (pedidos)
Pedidos->>Pedidos: 6 Crea detalle (detalle_pedido)
Pedidos->>Pagos: 7 Procesa pago (pagos_tb_baycodec)
Pagos-->>Pedidos: 8 Pago exitoso
Pedidos->>Inventario: 9 Descuenta stock (movimientos_inventario)
Inventario-->>Pedidos: 10 Inventario actualizado
Pedidos-->>Cliente: 11 Confirma orden creada con éxito

@enduml
```

Cliente -> Carrito: Añade items al carrito (carrito\_items)  
Cliente -> Pedidos: Solicita checkout / realizar pedido  
Pedidos -> Productos: Valida stock disponible (productos)  
Productos --> Pedidos: Stock confirmado  
Pedidos -> Pedidos: Crea registro principal (pedidos)  
Pedidos -> Pedidos: Crea detalle (detalle\_pedido)  
Pedidos -> Pagos: Procesa pago (pagos\_tb\_baycodec)  
Pagos --> Pedidos: Pago exitoso  
Pedidos -> Inventario: Descuenta stock (movimientos\_inventario)  
Inventario --> Pedidos: Inventario actualizado  
Pedidos --> Cliente: Confirma orden creada con éxito



## Diagrama de Actividades

@startuml

skinparam backgroundColor #FEFEFE

start

:Cliente selecciona productos y arma el carrito;

:Cliente solicita el proceso de Checkout;

if (¿Hay stock suficiente en productos?) then (sí)

:Aplicar cupones y calcular costos (envío, IGV);

:Crear registro en pedidos\_tb\_baycodec;

:Generar detalle del pedido (detalle\_pedido\_tb\_baycodec);

:Procesar pago (pagos\_tb\_baycodec);

if (¿El pago es exitoso?) then (sí)

:Registrar salida en movimientos\_inventario;

:Actualizar stock actual;

:Cambiar estado del pedido a "Pagado / En Preparación";

:Enviar notificación al cliente;

stop

else (no)

:Marcar pago como fallido;

:Cancelar o poner en espera el pedido;

stop

endif

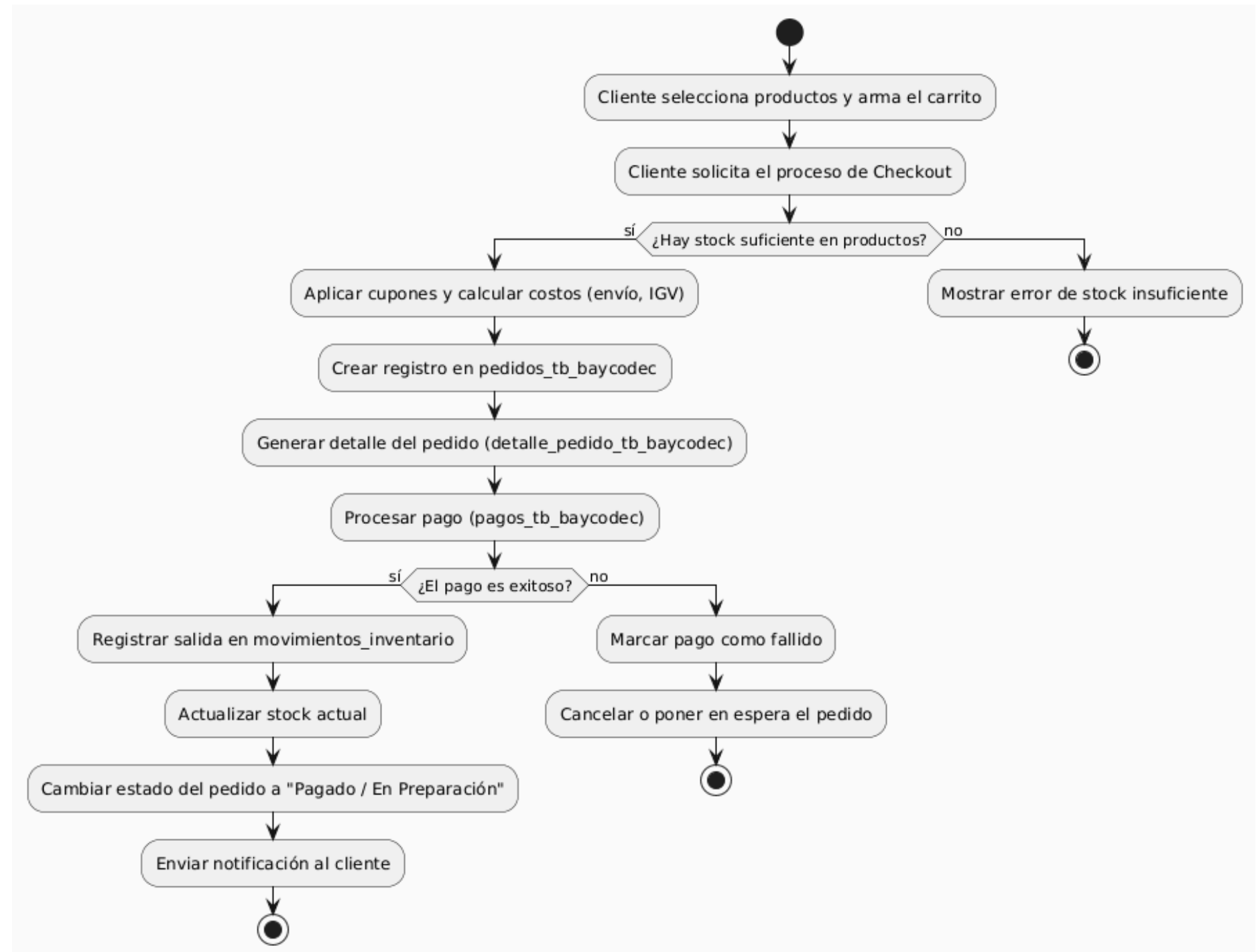
else (no)

:Mostrar error de stock insuficiente;

stop

endif

@enduml





## Diagrama de Estados (State Machine Diagram)

@startuml

skinparam backgroundColor #FEFEFE

[\*] --> Creado : Cliente confirma compra

Creado --> PendientePago : Generar registro de pago

PendientePago --> Pagado : Pago exitoso registrado

PendientePago --> Cancelado : Pago rechazado / Tiempo agotado

Pagado --> EnPreparacion : Vendedor empaqueta items

EnPreparacion --> Enviado : Despachado al servicio de entrega

Enviado --> Entregado : Cliente recibe la orden

Entregado --> [\*]

Cancelado --> [\*]

@enduml

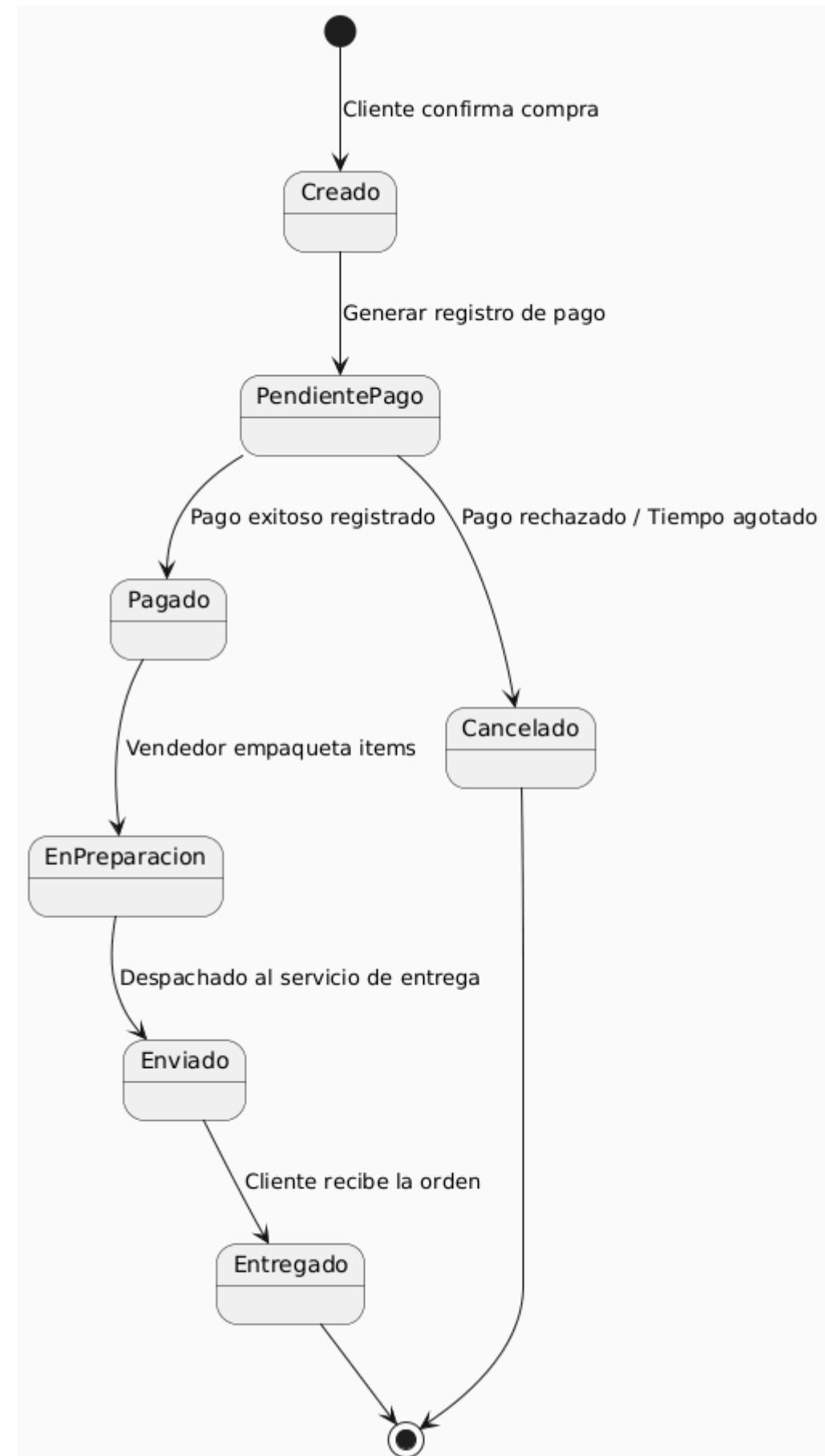


Diagrama de Paquetes (Package Diagram)

```
@startuml
skinparam backgroundColor #FEFEFE
skinparam packageStyle rectangle

package "Presentación / Vista" {
    [App Móvil / Web Client]
}

package "Controladores / Lógica de Negocio" {
    [Controlador Autenticación]
    [Controlador Carrito]
}

package "Modelo / Acceso a Datos (grafiluz_db)" {
    [clientes_tb_baycodec]
    [productos_tb_baycodec]
    [pedidos_tb_baycodec]
    [pagos_tb_baycodec]
    [movimientos_inventario_tb_baycodec]
}

[App Móvil / Web Client] --> [Controlador Carrito]
[Controlador Carrito] --> [Controlador Pedidos]
[Controlador Pedidos] --> [pedidos_tb_baycodec]
[Controlador Pedidos] --> [pagos_tb_baycodec]
[Controlador Pedidos] --> [movimientos_inventario_tb_baycodec]

@enduml
```

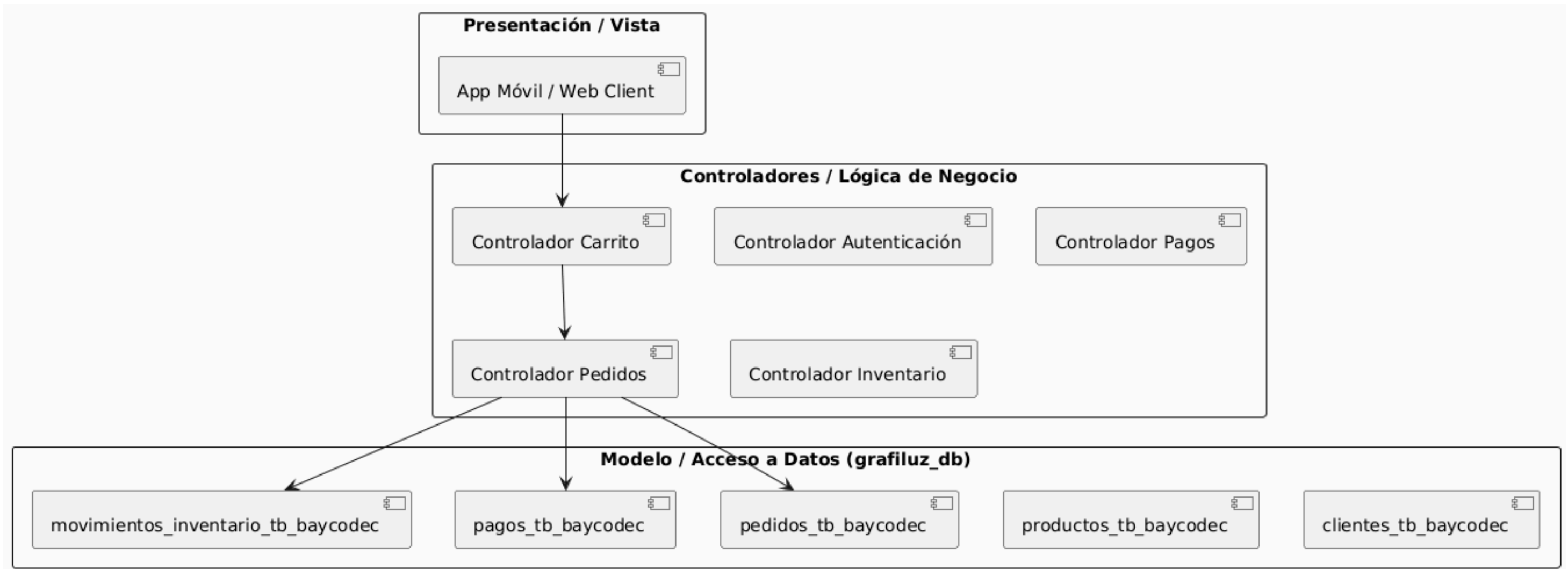


Diagrama de Objetos (Object Diagram)

```
@startuml
skinparam backgroundColor #FEFEFE

object "cliente1 : clientes_tb_baycodec" as cliente {
    id = 1
    nombre = "Juan Pérez"
    dni = "12345678"
    tipo_cliente = "B2C"
}

object "carrito1 : carritos_tb_baycodec" as carrito {
    id = 10
    cliente_id = 1
}

object "item1 : carrito_items_tb_baycodec" as item {
    id = 100
    carrito_id = 10
    producto_codigo = "PROD-001"
    cantidad = 2
}

object "producto1 : productos_tb_baycodec" as producto {
    id = 5
    codigo = "PROD-001"
    nombre = "Taza personalizada"
    precio_referencial_pen = 25.00
    stock = 50
}

cliente --> carrito : posee
carrito --> item : contiene
item --> producto : hace referencia a

@enduml
```

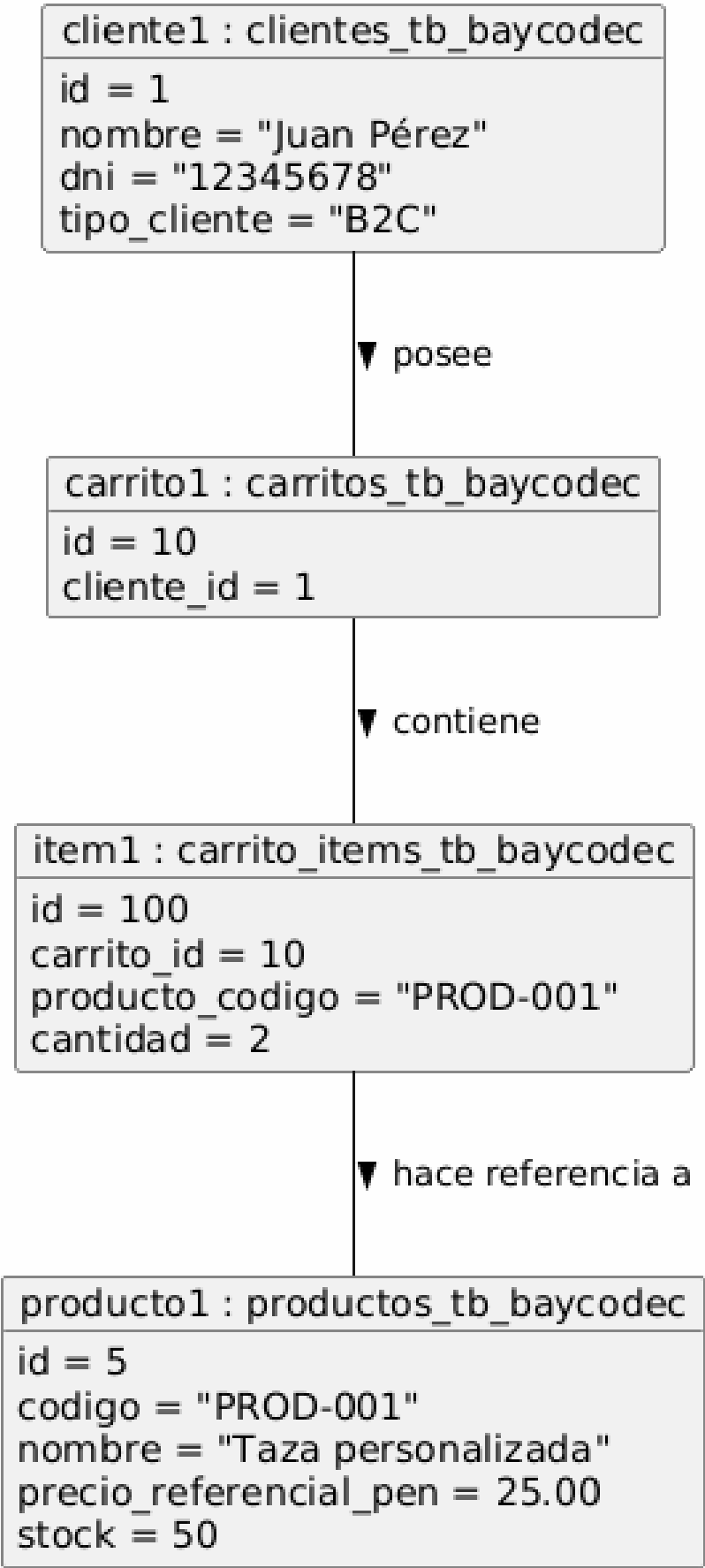


Diagrama de Comunicación (Communication Diagram)

```
@startuml
skinparam backgroundColor #FEFEFE
left to right direction

object "Cliente" as Cliente
object "carrito : carritos_tb_baycodec" as Carrito
object "pedido : pedidos_tb_baycodec" as Pedido
object "pago : pagos_tb_baycodec" as Pago
object "inventario : movimientos_inventario" as Inventario

Cliente --> Carrito : 1: Agregar items
Cliente --> Pedido : 2: Solicitar checkout
Pedido --> Pago : 3: Registrar pago
Pago --> Pedido : 4: Confirmar transacción
Pedido --> Inventario : 5: Descontar stock
@enduml
```

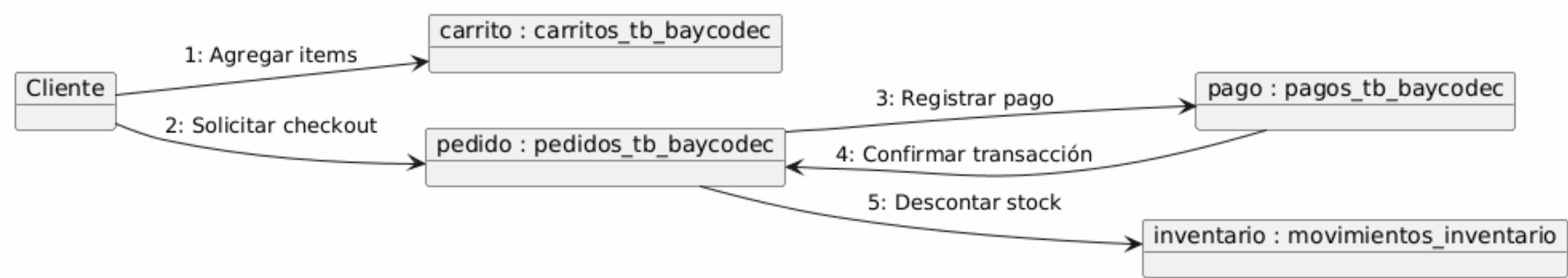
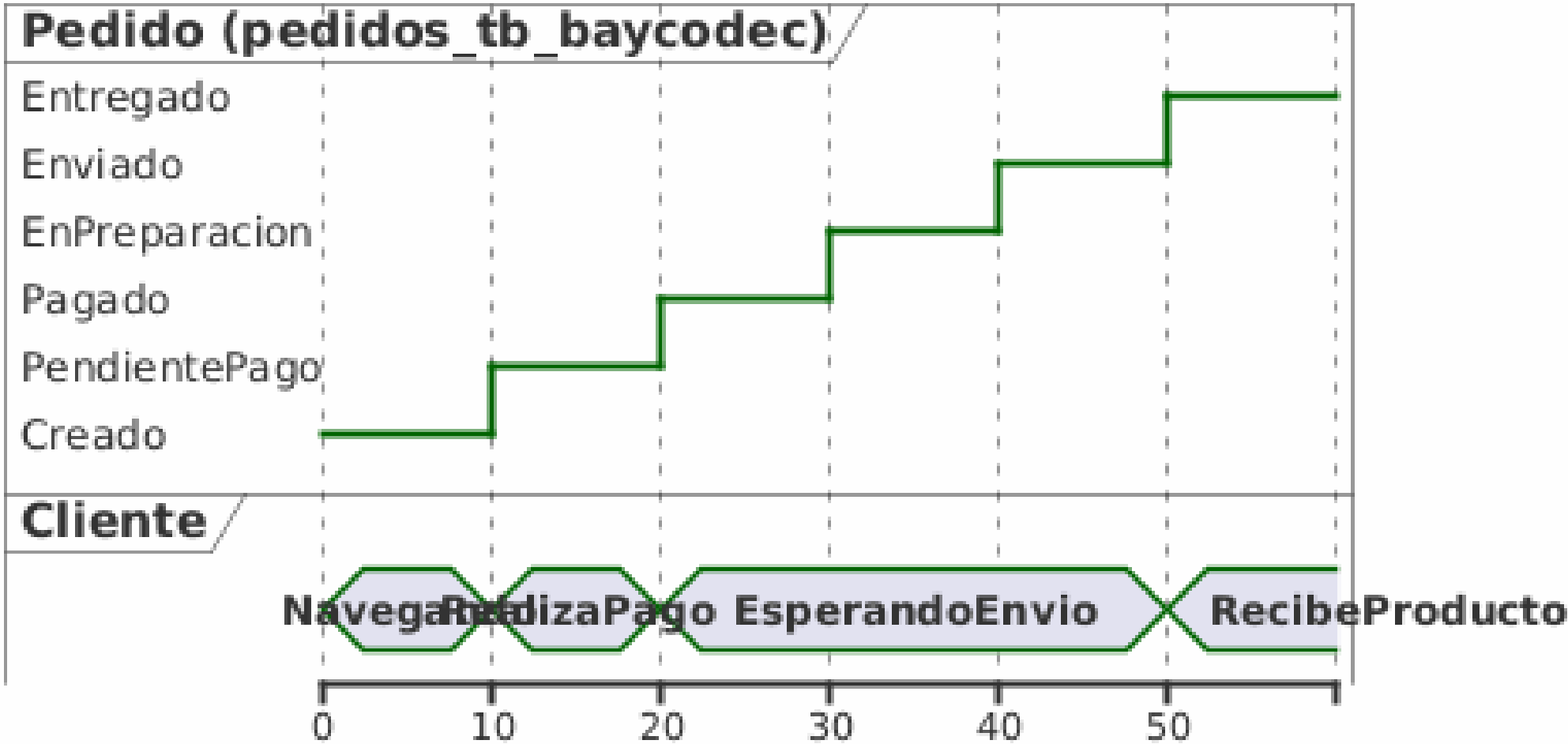


Diagrama de Tiempos (Timing Diagram)

@startuml  
skinparam backgroundColor #FEFEFE  
  
robust "Pedido (pedidos\_tb\_baycodec)" as Pedido  
concise "Cliente" as Cliente  
  
@0  
Cliente is Navegando  
Pedido is Creado  
  
@10  
Cliente is RealizaPago  
Pedido is PendientePago  
  
@20  
Cliente is EsperandoEnvio  
Pedido is Pagado  
  
@30  
Pedido is EnPreparacion  
  
@40  
Pedido is Enviado  
  
@50  
Cliente is RecibeProducto  
Pedido is Entregado  
  
@enduml



1. Diagrama de Clases (Class Diagram): Traducción de las tablas de la base de datos a clases UML con sus atributos, tipos de datos y relaciones de multiplicidad.
2. Diagrama de Componentes (Component Diagram): Organización del sistema en módulos lógicos (autenticación, catálogo, carrito, logística, finanzas, administración) y su conexión con la base de datos.
3. Diagrama de Despliegue (Deployment Diagram): Mapeo de la infraestructura de hardware y software, estructurada entre el dispositivo del cliente, el servidor backend y el servidor de base de datos MySQL.
4. Diagrama de Casos de Uso (Use Case Diagram): Funcionalidades y operaciones del sistema desde la perspectiva de los actores principales (**C**liente, **V**endedor, **A**ministrador).
5. Diagrama de Secuencia (Sequence Diagram): Interacción cronológica paso a paso entre los objetos durante el proceso de compra y validación de checkout.
6. Diagrama de Actividades (Activity Diagram): Flujo de trabajo lógico detallado con decisiones y validaciones para el procesamiento de pedidos y control de inventario.
7. Diagrama de Estados (State Machine Diagram): Ciclo de vida y transiciones de estado por las que pasa una entidad clave como el pedido (**p**edidos\_**t**b\_**b**aycode**c**).
8. Diagrama de Paquetes (Package Diagram): Agrupación modular del software en capas lógicas (Presentación, Controladores/Lógica de Negocio y Modelo/Acceso a Datos).
9. Diagrama de Objetos (Object Diagram): Instancias concretas con datos de ejemplo específicos (clientes, carritos, items, productos) en un momento determinado.
10. Diagrama de Comunicación (Communication Diagram): Red de objetos conectados espacialmente que intercambian mensajes numerados durante una transacción.
11. Diagrama de Tiempos (Timing Diagram): Evolución y restricciones temporales de los cambios de estado de los objetos a lo largo de un eje de tiempo.